Java File ProcessingA concise and systematic breakdown of core Java concepts including class loading, compilation, and runtime environments—tailored for efficient last-minute revision.

This compilation of handwritten Java notes captures the essential mechanisms of the Java platform, simplifying complex internals for rapid understanding and interview readiness.
An abstract overview of Java's execution lifecycle, highlighting key components like the JDK, JRE, JVM, and JIT compiler, aimed at enhancing conceptual clarity under time constraints.

- NOTES -

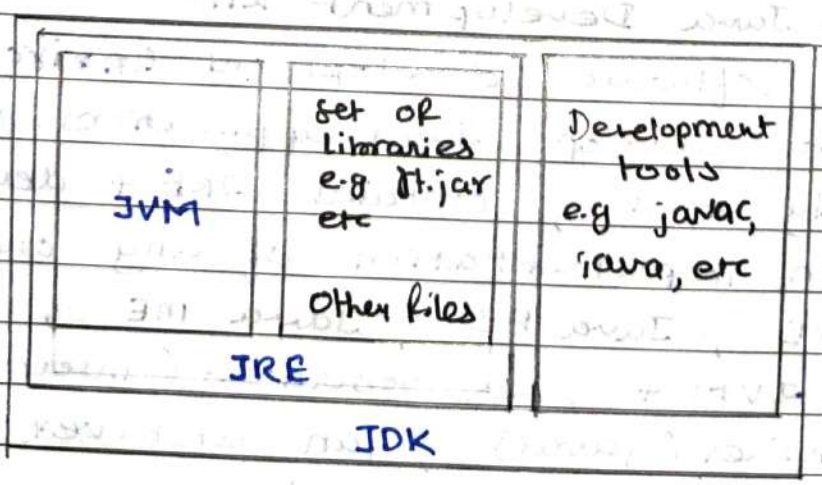
- JDK - Java Development kit
- It's a software development environment
- Used to develop Java applications & applets
- Physically exist, contains JRE + development tools.
- It is an implementation of any one below
  Java SE, Java EE, Java ME, Javafx
- JDK = JVM + other resources (interpreter / loader (java), a compiler (javac), an archiver (jar), a

document generator (Javadoc), etc.

- **JRE** - Java Runtime Environment
  - JRE is a set of software tools used for developing Java application.
  - Used to provide runtime environment
  - It physically exist ; It is the implementation of JVM
  - It consist set of libraries and other files that JVM uses at runtime.
- **JVM** - Java Virtual Machine
  - JVM is an abstract machine, doesn't physically exist.
  - Provides Runtime Environment for bytecode execution
  - JVM's are available for many hardware & software platforms

JDK, JRE, JVM are platform dependent because of differing OS configuration.

- 3 notions of JVM : specification, implementation, instance
- JVM performs following main tasks :
  - Loads code
  - Verifies code
  - Executes code
  - Provide Runtime Environment.



- Java File Processing
  - ↳ Compilation (through OS independent compiler)
  - ↳ Execution (in JVM which is custom built for every OS).

A) **Compilation :**

.java file ——————→ .class file
(in 7 steps)

i) Parse
- → Javac compiler reads your .java files.
- → It breaks code into parts (called tokens) and builds a tree structure (AST) to represent the code.

ii) Enter
- → The compiler records information about things like classes, methods, and variables in a symbol table (a kind of map for tracking definations).

iii) Process Annotations
- → If there are annotations (like @override, @Deprecated) the compiler processes them if needed.

iv) Attribute
- → Checks (compiler) the types, resolve naming convention.

v) Flow
- → data flow analysis like making sure variables are assigned before use, checking which parts of code can be reached.

vi) Desugar
- → Converts shortcut and advanced syntax like compler for-each loops, lambda into more basic Java code.

vii) Generate → generates .class file (bytecode) that JVM can run

Execution :

The class file generated are independent of the machine or the OS which allows them to run on any system.

→ First the main class file (main method) is passed to JVM, then it goes through (i) ClassLoader (ii) Byte code verifier (iii) JIT Compiler

- ClassLoader

→ main class is loaded into memory

→ All other class referenced through main are loaded through class loader.

→ ClassLoader, itself an object, creates a flat namespace as below

```
// loadClass function prototype
class r= loadClass(String className, boolean resolveIt);
```

className : Name of class to be loaded

resolveIt : flag to decide, referenced class should be loaded or not.

Two types of class Loader :

■ Primordial -

→ Also called Bootstrap ClassLoader, it loads core java classes from JDK like java.lang.string, java.util.list.

→ ~~Example~~ It is embedded into all JVMs and is default class loader.

Example : java.lang.Object

■ Non Primordial -

→ Is a user defined, which can be coded in order to customize the class loading process
→ It is defined and is preferred over default one

• Bytecode Verifier

→ Inspection on loaded class, so that instruction does not perform damage. Following are some checks:
- Variables are initialized before they are used
- Method calls match the type of object references.
- Rules for accessing private data & methods
- Local variable accesses fall within the runtime
- The run time stack does not overflow.
- If any check fails, class in not allowed to be loaded.

• Just In Time Compiler

→ Its job is to convert loaded bytecode into machine code, where bytecode is understandable by JVM and now native machine code can be ~~runned~~ ran directly by computer processor.
→ Faster program running because JIT ~~make~~ converts bytecode to machine code while program is running. Triggered only when program is running.